Report

v. 1.2

Customer

Shift DeFi



Smart Contract Audit

Shift DeFi Platform Update

17th June 2026

Report prepared by

ABDK Consulting

Contents

1 Changelog **5**

2 Introduction **6**

3 Project scope **7**

4 Methodology **10**

5 Our findings **11**

6 Moderate Issues **12**
CVF-1. INFO 12
CVF-2. FIXED 12
CVF-3. FIXED 13
CVF-4. INFO 13
CVF-5. FIXED 13
CVF-6. FIXED 14
CVF-7. FIXED 14
CVF-8. FIXED 14
CVF-9. INFO 14
CVF-10. INFO 15
CVF-11. FIXED 15
CVF-12. FIXED 15
CVF-13. FIXED 16

7 Recommendations **17**
CVF-14. INFO 17
CVF-15. INFO 17
CVF-16. INFO 17
CVF-17. INFO 18
CVF-18. INFO 18
CVF-19. INFO 18
CVF-20. INFO 18
CVF-21. INFO 19
CVF-22. INFO 19
CVF-23. INFO 19
CVF-24. INFO 19
CVF-25. INFO 20
CVF-26. INFO 20
CVF-27. INFO 20
CVF-28. INFO 20
CVF-29. INFO 21
CVF-30. INFO 21
CVF-31. INFO 21

CVF-32. INFO 21
CVF-33. INFO 22
CVF-34. INFO 22
CVF-35. INFO 22
CVF-36. INFO 22
CVF-37. INFO 23
CVF-38. INFO 23
CVF-39. INFO 23
CVF-40. INFO 23
CVF-41. INFO 24
CVF-42. INFO 24
CVF-43. INFO 24
CVF-44. INFO 24
CVF-45. INFO 25
CVF-46. INFO 25
CVF-47. INFO 25
CVF-48. INFO 25
CVF-49. INFO 26
CVF-50. INFO 26
CVF-51. INFO 26
CVF-52. INFO 26
CVF-53. INFO 27
CVF-54. INFO 27
CVF-55. INFO 27
CVF-56. INFO 27
CVF-57. INFO 28
CVF-58. INFO 28
CVF-59. INFO 28
CVF-60. INFO 28
CVF-61. INFO 29
CVF-62. INFO 29
CVF-63. INFO 29
CVF-64. INFO 29
CVF-65. INFO 30
CVF-66. INFO 30
CVF-67. INFO 30
CVF-68. INFO 30
CVF-69. INFO 31
CVF-70. INFO 31
CVF-71. INFO 31
CVF-72. INFO 32
CVF-73. INFO 32
CVF-74. INFO 32
CVF-75. INFO 32
CVF-76. INFO 33
CVF-77. INFO 33

CVF-78. INFO 33
CVF-79. INFO 33
CVF-80. INFO 34

1 Changelog

#	Date	Author	Description
0.1	29.12.25	A. Zveryanskaya	Initial Draft
0.2	29.12.25	A. Zveryanskaya	Minor revision
1.0	29.12.25	A. Zveryanskaya	Release
1.1	05.05.26	D. Khovratovich	Final
1.2	17.06.26	D. Khovratovich	More fixes

2 Introduction

All modifications to this document are prohibited. Violators will be prosecuted to the full extent of the U.S. law.

This document presents the results of a follow-up smart contract audit performed by ABDK Consulting for Shift, covering changes and improvements implemented in their DeFi platform following the initial audit. The review was conducted by Mikhail Vladimirov and Dmitry Khovratovich between 19th December and 25th December 2025. The goal of this audit was to evaluate the fixes applied to previously identified issues, assess newly introduced functionality, and identify any additional security concerns or code quality improvements in the modified codebase.

Our conclusion is that the development team has successfully addressed the major security vulnerabilities identified in the initial audit. The codebase has undergone substantial improvements, particularly in the strategy execution logic and price oracle infrastructure. The most significant enhancements include the introduction of a sophisticated state management system for strategies through `StrategyStateLib`, implementation of a modular price oracle aggregator architecture with support for both Chainlink and custom price feeds, and refinements to the harvest mechanism that enables reward collection without requiring full strategy allocation. The absence of major issues in this audit demonstrates the team's commitment to security and code quality.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 Project scope

This section provides a high-level overview of the Shift DeFi platform, a cross-chain yield aggregation protocol designed to enable users to deposit assets into a vault and earn returns through automated deployment across multiple blockchain networks and DeFi strategies. The System manages user deposits through a share-based tokenized vault architecture, coordinates asset allocation across different chains, executes yield-generating strategies, and facilitates cross-chain communication and bridging to maximize capital efficiency.

We were asked to review:

- Original Code

Files:

/			
	Vault.sol	SwapRouter.sol	StrategyTemplate.sol
	ReshufflingGateway.sol	PriceOracle Aggregator.sol	MessageRouter.sol
	ContainerPrincipal.sol	ContainerLocal.sol	ContainerAgent.sol
	BridgeAdapter.sol		
priceOracles/			
	CustomOracle Wrapper.sol	ChainlinkOracle Wrapper.sol	
libraries/			
	StrategyStateLib.sol		
libraries/helpers/			
	RingCacheLibrary.sol		

The System is implemented in Solidity version 0.8.28, utilizing the latest language features and compiler optimizations available at the time of development. The codebase employs OpenZeppelin's battle-tested upgradeable contract libraries extensively, including upgradeable versions of access control mechanisms, ERC20 token standards, reentrancy protection, and proxy initialization patterns. This choice enables the System to support future protocol upgrades while maintaining state continuity. The development approach follows modern Solidity best practices, including comprehensive use of SafeERC20 for token transfers, the Math library for precision arithmetic operations, and EnumerableSet for efficient collection management.

At the architectural core lies `Vault`, which serves as the primary user interaction point. Users deposit a designated notion token into the vault and receive proportional shares represented as ERC20 tokens. The vault implements a batch processing mechanism for both deposits and withdrawals, aggregating multiple user requests to optimize gas costs and enable efficient cross-chain coordination. The vault maintains connections to multiple containers, each assigned specific weight allocations that determine capital distribution. Container weights are configurable by authorized roles, allowing dynamic rebalancing of capital across different deployment targets. The vault tracks net asset values at multiple stages of the deposit and withdrawal lifecycle, ensuring accurate share price calculations that reflect the underlying value of deployed assets.

The container layer provides three distinct implementations addressing different deployment scenarios. `ContainerLocal` handles strategies deployed on the same blockchain as the vault, providing a straightforward execution path without cross-chain complexity. `ContainerPrincipal` and `ContainerAgent` work together to enable cross-chain deployments, where the principal resides on the vault's chain and coordinates with agent containers on remote chains. These cross-chain containers implement a message-passing protocol for deposit and withdrawal requests, ensuring synchronized state transitions across networks. Each container manages whitelisted tokens, implements role-based access control for operational activities, and maintains connections to swap routing infrastructure for token conversions.

Strategy execution is abstracted through `StrategyTemplate`, which defines a common interface for entering and exiting positions, calculating allocated net asset values, and handling emergency withdrawals. The System includes a concrete implementation for Compound V3 lending markets, demonstrating integration with external yield protocols. Strategies define input and output tokens, track state through identifiers, and implement state-specific exit logic. The template pattern enables strategy containers to manage multiple strategies simultaneously, coordinating entry and exit operations through bitmask tracking mechanisms. Strategy containers calculate aggregate net asset values across all deployed strategies, collect performance fees for the treasury, and support emergency exit capabilities for risk mitigation.

Cross-chain communication infrastructure centers on `MessageRouter`, which manages message delivery between containers across different blockchains. The router implements path-based whitelisting, connecting sender addresses and destination chains to specific receiver contracts. Nonce tracking prevents replay attacks by ensuring messages are processed in order. The router supports multiple message adapter integrations, allowing the System to leverage different cross-chain communication protocols. Bridge operations are abstracted through `BridgeAdapter`, with a concrete implementation for Across V3 protocol. Bridge adapters maintain path mappings between tokens on source and destination chains, validate bridge instructions against configured parameters, and implement claimable amount tracking for asynchronous bridge completion.

Token swapping capabilities are provided through `SwapRouter`, which routes swap requests to whitelisted adapter contracts. The System implements adapters for Uniswap V3 and 1inch aggregator, enabling strategies to convert rewards and rebalance token positions. Additionally, `UniversalAdapter` provides a flexible generic swap adapter that can be configured to work with various DEX protocols through customizable swap info parameters. The swap router validates minimum output amounts, transfers tokens

securely, and emits events for off-chain monitoring. Price oracle functionality relies on Chainlink data feeds through `PriceOracle`, which provides USD valuations for net asset value calculations. The oracle maintains mappings from token addresses to Chainlink feed contracts, retrieves latest prices with decimal precision, and converts token amounts to unified USD denominations for consistent accounting across different tokens.

Additional infrastructure includes `ReshufflingGateway` for managed container reconfigurations, supporting both reshuffling operations that migrate capital between containers and repair operations for emergency recovery scenarios. The gateway coordinates with the vault to claim assets, perform necessary token swaps, and redistribute capital according to updated allocations. `Collector` provides a simple claiming interface for users to retrieve accepted notion tokens from processed batches. Custom libraries throughout the codebase handle message encoding and decoding, address array packing for efficient cross-chain transmission, ring cache implementations for deduplication, and helper utilities for decimal normalization and error definitions.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment**. The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis**. Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis**. For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis**. The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** cover code style, best practices and general improvements.

5 Our findings

We've provided the client with a few recommendations.

	Info	Fixed
Moderate	13	9

6 Moderate Issues

CVF-1. INFO

- **Category** Suboptimal
- **Source** RingCacheLibrary.sol

Description Currently, there is no relation between cache storage address and cache ID. This allows having several caches with the same ID, consumes storage space for storing cache ID, and requires cache ID to be passed as an argument along with the cache storage address.

Recommendation Make a cache storage address to be a function of the cache ID, like a hash of the cache ID with some salt. This would allow identifying caches solely by their IDs, so cache storage addresses will be calculated inside the library.

```
25  +function initialize(RingCache storage _cache, bytes32 _id, uint256
        ↪ _cacheSize) internal {
```

CVF-2. FIXED

- **Category** Procedural
- **Source** MessageRouter.sol

Recommendation The events should be emitted inside the ting cache library, as all necessary information is available there.

```
181 217  _removeFromCache(chainTo, messageWithPathAndNonce);
218  +emit MessageRemovedFromCache(_sendMessagesCache.id, nonce, chainTo,
         ↪  path);
229  +require(_sendMessagesCache.exists(cachedKey), RingCacheLibrary.
         ↪ DoesNotExists(_sendMessagesCache.id, cachedKey));
231  +emit RingCacheLibrary.CacheEvicted(_sendMessagesCache.id, cachedKey
         ↪ );
```

CVF-3. FIXED

- **Category** Suboptimal
- **Source** PriceOracleAggregator.sol

Description This function does normalization several times.

Recommendation Calculate value0*price9/price1 without normalization, and then normalize the result at once taking into account decimals of tokens and prices. This would also optimize a quite common case where decimals0 and decimals1 are the same.

```
53 +function getRelativeValueUnified(address token0, address token1,
      ↪ uint256 value0) external view returns (uint256) {
```

CVF-4. INFO

- **Category** Procedural
- **Source** PriceOracleAggregator.sol

Description In ERC-20 the “decimals” property is used by UI to render token amounts in human-readable way. Using this property in smart contract is discouraged.

Recommendation Treat all token amounts as integers.

```
78 +function _normalizePrice(uint256 price, uint8 decimals) internal
      ↪ pure returns (uint256) {
```

CVF-5. FIXED

- **Category** Unclear behavior
- **Source** StrategyTemplate.sol

Description This expensive call should be executed only when ”msg.sender“ is not equal to “strategyContainerCached”.

```
56 +bool isHarvestManager = AccessControlUpgradeable(
      ↪ strategyContainerCached).hasRole(
57 +    HARVEST_MANAGER_ROLE,
58 +    msg.sender
59 +);
```

```
66 +bool isEmergencyManager = AccessControlUpgradeable(
      ↪ strategyContainerCached).hasRole(
67 +    EMERGENCY_MANAGER_ROLE,
68 +    msg.sender
69 +);
```

CVF-6. FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation Should e “>=”.

```
207 +vars.stateToNavAfterEnter > vars.stateToNavBeforeEnter +
       ↪ minNavDelta,
```

CVF-7. FIXED

- **Category** Bad naming
- **Source** StrategyTemplate.sol

Description The error name doesn’t match actual condition checked.

Recommendation Use different error name.

```
235 +require(vars.toStateBitmask != 0, Errors.ZeroAmount());
```

```
365 +require(vars.toStateBitmask != 0, Errors.ZeroAmount());
```

CVF-8. FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation Should be “>=”.

```
253 +vars.stateToNavAfterEnter > vars.stateToNavBeforeEnter +
       ↪ minNavDelta,
```

CVF-9. INFO

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation It could also be useful to specify the minimum token output amounts.

```
295 +uint256 maxNavDelta
```

CVF-10. INFO

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Description Here two external calls to the same contract are made sequentially.

Recommendation Merge them into a single call to save gas.

```
341 +vars.treasury = IStrategyContainer(_strategyContainer).treasury();
342 +vars.feePct = IStrategyContainer(_strategyContainer).feePct();
```

CVF-11. FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation The second conditional operator is redundant. Just do:
if (balance > 0) { ... vars.hasReminder = true; }

```
437 +if (balance > 0) {
438 +    IERC20(vars.tokens[i]).forceApprove(vars.container, balance);
439 +    vars.amounts[i] = balance;
264 440  }
```

```
442 +if (balance > 0 && !vars.hasRemainder) {
443 +    vars.hasRemainder = true;
444 +}
```

CVF-12. FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation The second conditional statement is redundant.Just do: if (amount > 0) { ... vars.hasRemainder = true; }

```
468 +if (amount > 0) {
469 +    IERC20(vars.outputTokens[i]).forceApprove(vars.container,
    ↪ amount);
470 +    vars.outputAmounts[i] = amount;
287 471  }
```

```
473 +if (vars.outputAmounts[i] > 0 && !vars.hasRemainder) {
474 +    vars.hasRemainder = true;
475 +}
```

CVF-13. FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Description Here any value of “share” greater than “BPS” is treated as “BPS”, which can hide possible errors.

Recommendation Explicitly require “share <= BPS”.

```
489 +if (share < BPS) {
```

```
491 +} else {
492 +    amounts[i] = balance;
```

7 Recommendations

CVF-14. INFO

- **Category** Procedural
- **Source** ChainlinkOracleWrapper.sol

Description Consider specifying as “^0.8.0” unless there is something special regarding this particular version.

Recommendation Also relevanr for: CustomOracleWrapper.sol, PriceOracleAggregator.sol.

```
2 +pragma solidity ^0.8.28;
```

CVF-15. INFO

- **Category** Procedural
- **Source** ChainlinkOracleWrapper.sol

Description We didn’t review these files.

```
8 +import {IChainlinkPriceFeed} from "../dependencies/interfaces/
     ↪ chainlink/IChainlinkPriceFeed.sol";
9 +import {IChainlinkOracleWrapper} from "../interfaces/
     ↪ IChainlinkOracleWrapper.sol";
```

CVF-16. INFO

- **Category** Bad datatype
- **Source** ChainlinkOracleWrapper.sol

Recommendation The type for this mapping should be: mapping (IERC20 ⇒ IChainlinePriceFeed)

```
14 +mapping(address => address) public tokenToChainlinkFeed;
```

CVF-17. INFO

- **Category** Bad datatype
- **Source** ChainlinkOracleWrapper.sol

Recommendation The type for the “token” argument should be “IERC20”.

```
21 +function setChainlinkFeed(address token, address feed) external
      ↪ override onlyRole(ORACLE_MANAGER_ROLE) {
```

```
30 +function getPrice(address token) external view override returns (
      ↪ uint256, uint8) {
```

CVF-18. INFO

- **Category** Bad datatype
- **Source** ChainlinkOracleWrapper.sol

Recommendation The type for the “food” argument should be “IChainlinkPriceFeed”.

```
21 +function setChainlinkFeed(address token, address feed) external
      ↪ override onlyRole(ORACLE_MANAGER_ROLE) {
```

CVF-19. INFO

- **Category** Suboptimal
- **Source** ChainlinkOracleWrapper.sol

Recommendation This check is redundant, as it is anyway possible to pass a dead token address.

```
22 +require(token != address(0), Errors.ZeroAddress());
```

CVF-20. INFO

- **Category** Bad naming
- **Source** ChainlinkOracleWrapper.sol

Description The semantics of the returned value is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
30 +function getPrice(address token) external view override returns (
      ↪ uint256, uint8) {
```

CVF-21. INFO

- **Category** Suboptimal
- **Source** ChainlinkOracleWrapper.sol

Recommendation This check is redundant, as it is superseded by the following “chainlinkFeed” test.

```
31 +require(token != address(0), Errors.ZeroAddress());
```

CVF-22. INFO

- **Category** Procedural
- **Source** CustomOracleWrapper.sol

Description We didn’t review these files.

```
5 +import {IPriceOracle} from "../interfaces/IPriceOracle.sol";
6 +import {ICustomOracleWrapper} from "../interfaces/
    ↪ ICustomOracleWrapper.sol";
```

CVF-23. INFO

- **Category** Bad datatype
- **Source** CustomOracleWrapper.sol

Recommendation The key type for this mapping should be “IERC20”.

```
14 +mapping(address => uint256) public tokenToPrice;
```

CVF-24. INFO

- **Category** Documentation
- **Source** CustomOracleWrapper.sol

Description The number format for the price is unclear.

Recommendation Explain in a documentation comment.

```
14 +mapping(address => uint256) public tokenToPrice;
```

CVF-25. INFO

- **Category** Bad datatype
- **Source** CustomOracleWrapper.sol

Recommendation The type fro the “token” argument should be “IERC20”.

```
29 +function submitPrice(address token, uint256 price) external
      ↪ onlyRole(FEEDER_ROLE) {
```

```
34 +function getPrice(address token) external view returns (uint256,
      ↪ uint8) {
```

CVF-26. INFO

- **Category** Documentation
- **Source** CustomOracleWrapper.sol

Description The number format for the “price” argument is unclear.

Recommendation Explain in a documentation comment.

```
29 +function submitPrice(address token, uint256 price) external
      ↪ onlyRole(FEEDER_ROLE) {
```

CVF-27. INFO

- **Category** Documentation
- **Source** CustomOracleWrapper.sol

Description Semantics of the returned values is unclear.

Recommendation Explain in a documentation comment.

```
34 +function getPrice(address token) external view returns (uint256,
      ↪ uint8) {
```

CVF-28. INFO

- **Category** Procedural
- **Source** StrategyStateLib.sol

Description Consider specifying as “^.8.0” unless there is something special regarding this particular version.

```
3 +pragma solidity ^0.8.28;
```

CVF-29. INFO

- **Category** Bad naming
- **Source** StrategyStateLib.sol

Description The word “position” here is to generic and it is unclear, what kind of position is ment.

Recommendation Rename to “XXX_STATE_BIT” or something like this.

```
6  +uint256 constant TOKEN_STATE_POSITION = 8;
7  +uint256 constant PROTOCOL_STATE_POSITION = 9;
8  +uint256 constant TARGET_STATE_POSITION = 10;
```

CVF-30. INFO

- **Category** Suboptimal
- **Source** StrategyStateLib.sol

Recommendation This error could be made more useful by adding certain parameters into it.

```
15 +error InconsistentState();
```

CVF-31. INFO

- **Category** Procedural
- **Source** StrategyStateLib.sol

Recommendation Brackets are redundant.

```
19 +return (state & PROTOCOL_STATE_MASK) != 0;
```

```
22 +return (state & TOKEN_STATE_MASK) != 0;
```

```
26 +return (state & TARGET_STATE_MASK) != 0;
```

CVF-32. INFO

- **Category** Suboptimal
- **Source** StrategyStateLib.sol

Recommendation This condition could be simplified as: !(_isTargetState || _isProtocol-State || _isTokenState).

```
42 +if (!_isTargetState && !_isProtocolState && !_isTokenState) {
```

CVF-33. INFO

- **Category** Bad naming
- **Source** StrategyStateLib.sol

Description The error name is confusing, as there is no zero value here.

Recommendation Rename to “VoidState” or “NullState” or “EmptyState”.

```
43 +revert ZeroState();
```

CVF-34. INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for the “strategy” argument should be more specific.

```
39 +address strategy,
```

```
48 +address strategy
```

CVF-35. INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for these argument should be “IERC20[]”.

```
40 +address[] calldata inputTokens,
41 +address[] calldata outputTokens
```

CVF-36. INFO

- **Category** Readability
- **Source** ContainerAgent.sol

Recommendation Should be “else if”.

```
325 +if (
```

```
332 +if (
```

CVF-37. INFO

- **Category** Readability
- **Source** ContainerAgent.sol

Recommendation Should be “else revert”.

```
340 +revert Errors.IncorrectContainerStatus();
```

CVF-38. INFO

- **Category** Bad datatype
- **Source** ContainerLocal.sol

Recommendation The type for these arguments should be more specific.

```
83 +address strategy,
```

```
92 +address strategy
```

CVF-39. INFO

- **Category** Bad datatype
- **Source** ContainerLocal.sol

Recommendation The type for these argument should be “IERC20[]”.

```
84 +address[] calldata inputTokens,
85 +address[] calldata outputTokens
```

CVF-40. INFO

- **Category** Readability
- **Source** ContainerLocal.sol

Recommendation Should be “else if”.

```
178 +if (
```

```
185 +if (
```

CVF-41. INFO

- **Category** Readability
- **Source** ContainerLocal.sol

Recommendation Should be “else revert”.

```
192 +revert Errors.IncorrectContainerStatus();
```

CVF-42. INFO

- **Category** Documentation
- **Source** MessageRouter.sol

Description The particular format of the “path” argument is unclear.

Recommendation Explain in a documentation comment,

```
48 +function encodeMessage(uint256 nonce, bytes32 path, bytes memory
   ↪ message) public pure returns (bytes memory) {
49 +    return abi.encodePacked(nonce, path, message);
```

CVF-43. INFO

- **Category** Readability
- **Source** ContainerPrincipal.sol

Recommendation Should be “else if”.

```
191 +if (messageType == Codec.WITHDRAWAL_RESPONSE_TYPE) {
```

CVF-44. INFO

- **Category** Readability
- **Source** ContainerPrincipal.sol

Recommendation Should be “else revert” for readability, otherwise it looks like the function always reverts.

```
201 +revert Codec.WrongMessageType(messageType);
```

CVF-45. INFO

- **Category** Suboptimal
- **Source** PriceOracleAggregator.sol

Recommendation Consider grouping Open Zeppelin imports together.

```
4  +import {Initializable} from "@openzeppelin/contracts-upgradeable/
       ↪ proxy/utils/Initializable.sol";
5  +import {AccessControlUpgradeable} from "@openzeppelin/contracts-
       ↪ upgradeable/access/AccessControlUpgradeable.sol";
```

```
8  +import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
```

CVF-46. INFO

- **Category** Procedural
- **Source** PriceOracleAggregator.sol

Description We didn't review this file.

```
10 +import {IPriceOracleAggregator} from "./interfaces/
       ↪ IPriceOracleAggregator.sol";
```

CVF-47. INFO

- **Category** Bad naming
- **Source** PriceOracleAggregator.sol

Description The semantics of the keys and values in this mapping is unclear.

Recommendation Give descriptive names to the names and values and/or explain in a documentation comment.

```
21 +mapping(address => address) public override priceOracles;
```

CVF-48. INFO

- **Category** Bad datatype
- **Source** PriceOracleAggregator.sol

Recommendation The key type for this mapping should be "IERC20".

```
21 +mapping(address => address) public override priceOracles;
```

CVF-49. INFO

- **Category** Bad datatype
- **Source** PriceOracleAggregator.sol

Recommendation The value type for this mapping should be “IPriceOracle”.

```
21 +mapping(address => address) public override priceOracles;
```

CVF-50. INFO

- **Category** Bad datatype
- **Source** PriceOracleAggregator.sol

Recommendation The type for the “token” argument should be “IERC20”.

```
34 +function setPriceOracle(address token, address priceOracle)
      ↪ external override onlyRole(ORACLE_MANAGER_ROLE) {
```

```
43 +function fetchTokenPrice(address token) external view override
      ↪ returns (uint256) {
```

CVF-51. INFO

- **Category** Bad datatype
- **Source** PriceOracleAggregator.sol

Recommendation The type for the “priceOracle” argument should be “IPriceOracle”.

```
34 +function setPriceOracle(address token, address priceOracle)
      ↪ external override onlyRole(ORACLE_MANAGER_ROLE) {
```

CVF-52. INFO

- **Category** Suboptimal
- **Source** PriceOracleAggregator.sol

Recommendation This check is redundant, as it is anyway possible to pass a dead token address.

```
35 +require(token != address(0), Errors.ZeroAddress());
```

CVF-53. INFO

- **Category** Documentation
- **Source** PriceOracleAggregator.sol

Description The number format of the returned price is unclear.

Recommendation Explain in a documentation comment.

```
43 +function fetchTokenPrice(address token) external view override
   ↪ returns (uint256) {
```

CVF-54. INFO

- **Category** Suboptimal
- **Source** PriceOracleAggregator.sol

Recommendation This check is redundant, as it is superseded by the following “priceOracle” check.

```
44 +require(token != address(0), Errors.ZeroAddress());
```

CVF-55. INFO

- **Category** Bad datatype
- **Source** PriceOracleAggregator.sol

Recommendation The type for the token arguments should be “IERC20”.

```
53 +function getRelativeValueUnified(address token0, address token1,
   ↪ uint256 value0) external view returns (uint256) {
```

CVF-56. INFO

- **Category** Documentation
- **Source** PriceOracleAggregator.sol

Description The number format for the returned value is unclear.

Recommendation Explain in a documentation comment.

```
53 +function getRelativeValueUnified(address token0, address token1,
   ↪ uint256 value0) external view returns (uint256) {
```

CVF-57. INFO

- **Category** Suboptimal
- **Source** PriceOracleAggregator.sol

Description This check optimizes a rare use case (zero amount) while making common case less efficient.

Recommendation Consider removing this optimization.

```
56 +if (value0 == 0) return 0;
```

CVF-58. INFO

- **Category** Procedural
- **Source** StrategyTemplate.sol

Description We didn't review this file.

```
18 +import {IPriceOracleAggregator} from "./interfaces/
   ↪ IPriceOracleAggregator.sol";
```

CVF-59. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The type for this variable should be "IStrategyContainer".

```
33 +address internal _strategyContainer;
```

CVF-60. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The type for this variable should be "IERC20".

```
34 +address internal _notion;
```

CVF-61. INFO

- **Category** Documentation
- **Source** StrategyTemplate.sol

Description Semantics of the keys and values in this mappings is unclear.

Recommendation Gove descriptive names to the key and value and/or explain in a documentation comment.

```
38 +mapping(bytes32 => uint256) private _stateBitmasks;
```

CVF-62. INFO

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation This check is redundant, as it is superseded by the “msg.sender” check in the next line.

```
49 +require(_strategyContainer != address(0), Errors.ZeroAddress());
```

CVF-63. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The return type should be “IERC20”.

```
92 +function getInputTokens() external view override returns (address[]
   ↪  memory) {
```

```
96 +function getOutputTokens() external view override returns (address
   ↪ [] memory) {
```

CVF-64. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The argument type should be “IStrategyContainer”.

```
112 +function __StrategyTemplate_init(address strategyContainer)
   ↪ internal onlyInitializing {
```

CVF-65. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The argument type should be “IERC20[]”.

```
123 +function setInputTokens(address[] calldata inputTokens) external
       ↪ override onlyStrategyContainer {
```

```
135 +function setOutputTokens(address[] calldata outputTokens) external
       ↪ override onlyStrategyContainer {
```

CVF-66. INFO

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation These checks are redundant, as it is anyway possible to pass a dead token address. Remove these checks.

```
126 +require(inputTokens[i] != address(0), Errors.ZeroAddress());
```

```
138 +require(outputTokens[i] != address(0), Errors.ZeroAddress());
```

CVF-67. INFO

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Recommendation This check should be dome earlier.

```
156 +if (isTargetState) {
157 +    require(_targetStateId == bytes32(0), TargetStateAlreadySet());
```

CVF-68. INFO

- **Category** Documentation
- **Source** StrategyTemplate.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
166 +) external payable onlyStrategyContainer nonReentrant returns (
       ↪ uint256, bool, uint256[] memory) {
```

CVF-69. INFO

- **Category** Documentation
- **Source** StrategyTemplate.sol

Description The semantics of the retuned value is unclear.

Recommendation Give descriptive name to the returned value and/or explain in a documentation comment.

```
262 +function acceptNav(bytes32 acceptedStateId) public
       ↪ onlyEmergencyManager returns (uint256) {
```

CVF-70. INFO

- **Category** Documentation
- **Source** StrategyTemplate.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
296 +) external payable onlyStrategyContainer nonReentrant returns (
       ↪ address[] memory, uint256[] memory) {
```

```
426 +function _prepareFundsAfterEnter() private returns (uint256[]
       ↪ memory, bool) {
```

```
275 455  ) private returns (uint256[] memory, bool) {
```

CVF-71. INFO

- **Category** Procedural
- **Source** StrategyTemplate.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
387 +bytes32[] calldata toStateIds,
388 +uint256[] calldata shares
```

CVF-72. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The type for the “token” argument should be “IERC20”.

```
406 +function getTokenAmountInNotion(address token, uint256 amount)
      ↪ public view returns (uint256) {
```

CVF-73. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The type for the “tokens” argument should be “IERC20[]”.

```
483 +function _tokensAmountsDump(address[] memory tokens, uint256 share)
      ↪  private view returns (uint256[] memory) {
```

CVF-74. INFO

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The type for the “tokenIn” argument should be “IERC20”.

```
502 +function _swapToInputTokens(address tokenIn, address tokenOut)
      ↪ internal virtual {
```

CVF-75. INFO

- **Category** Documentation
- **Source** SwapRouter.sol ONLY YELLOW)

Description The semantics of the keys and values in this mappings is unclear.

Recommendation Give descriptive names to the key and value and/or explain in a documentation comment.

```
20 +mapping(address => mapping(address => PredefinedSwapParameters))
      ↪ public predefinedSwapParameters;
```

CVF-76. INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol ONLY YELLOW)

Recommendation The type for he keys in this mapping should be “IERC20”.

```
20 +mapping(address => mapping(address => PredefinedSwapParameters))
      ↪ public predefinedSwapParameters;
```

CVF-77. INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol ONLY YELLOW)

Recommendation The type for these arguments should be “IERC20”.

```
51 +address tokenIn,
52 +address tokenOut,
```

```
65 +address tokenIn,
66 +address tokenOut,
```

CVF-78. INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol ONLY YELLOW)

Recommendation The type for this argument should be more specific.

```
53 +address adapter,
```

CVF-79. INFO

- **Category** Suboptimal
- **Source** SwapRouter.sol ONLY YELLOW)

Recommendation These checks are redundant, as it is anyway possible to pass dead token address.

```
56 +require(tokenIn != address(0), Errors.ZeroAddress());
57 +require(tokenOut != address(0), Errors.ZeroAddress());
```

CVF-80. INFO

- **Category** Documentation
- **Source** SwapRouter.sol ONLY YELLOW)

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
69 +) external payable returns (bool, uint256) {
```



ABDK
Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting